American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Final-Term Project Report

Summer 2025-2026

Section:B

Group:07

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: MD. Tanvir Islam Sarker Tamim ID: 23-50786-1	BERT variant comparison, overall model performance comparison, Ensemble result analysis, and final report contribution.
2.	Name: Tanvir Mahatab Anan ID: 23-50807-1	Implementing DistilBERT, Preprocessing, Ensemble, Python environment creation.
3.	Name: Jannatun Nesa Jerin ID: 23-50798-1	BERT implementation, hyperparameter experimentation, model evaluation, result analysis, and final report preparation
4.	Name: MD. Mahmodul Hasan ID: 23-50794-1	BERT variant research, model selection analysis, result collection, and final report contribution

A Comparative Analysis of Traditional Machine Learning and Transformer Models for Cross-Domain Text Classification

MD. Tanvir Islam Sarker Tamim, Tanvir Mahatab Anan, Jannatun Nesa Jerin, MD. Mahmodul Hasan
American International University-Bangladesh
Dhaka, Bangladesh
23-50786-1@student.aiub.edu, 23-50807-1@student.aiub.edu, 23-50798-1@student.aiub.edu, 23-50794-1@student.aiub.edu

1 Introduction

Natural Language Processing (NLP) is a significant branch of Artificial Intelligence that assists computers to comprehend and extract meaning out of human language. **Text Classification** is one of the popular tasks in NLP, in which text files are labelled as belonging to various categories according to their content. This project is aimed at comparing various Machine Learning and Deep Learning models to text-classify.

Two different datasets are used in this project. The first dataset will be **AG News Topic Classification Dataset** which consists of news items in four categories: World, Sports, Business, and Sci/Tech. The second data is the **MSME Legal Dispute Classification Dataset** which comprises documents of legal disputes that are categorized into four categories that are chosen. These datasets possess various forms of text and this makes them relevant in determining the performance of various NLP models.

During the midterm section, three classic algorithms of Machine Learning were used: **Naive Bayes, Logistic Regression, and Support Vector Machine (SVM)**. The conversion of text to numerical features was done using two methods of text representation, **Bag of Words (BoW)** and **TF-IDF**. **Accuracy, Precision, Recall, and F1 Score** were used in the evaluation of the models.

Finally, transformers models, such as **BERT and DistilBERT**, were trial run in text classification in the final-term section. Diverse **learning rate, batch size, and weight decay** values were experimented with to find out the most effective set-up of every model. Last, an **Ensemble Model** was created which is a combination of predictions of various models and this is done to attain a higher overall classification performance.

The primary objective of the project is to find out the similarities between the traditional Machine learning models and the modern Transformer-based models and the approach that is more effective on the two chosen datasets. The experimental findings give a direct insight into the functionality and shortcomings of various NLP models to classify texts.

2 Midterm Result Analysis

Algorithm	Representation	Dataset 1	Dataset 2
Naive Bayes	**BoW**	Accuracy: 0.825 Precession: 0.827 Recall: 0.825 **F1 Score: 0.823**	Accuracy: 0.8325 Precession: 0.8301 Recall: 0.8325 **F1 Score: 0.8280**
	TF-IDF	Accuracy: 0.763 Precession: 0.8001 Recall: 0.763 F1 Score: .754	Accuracy: 0.6775 Precession: 0.7106 Recall: 0.6775 F1 Score: 0.6248
Logistic Regression	**BoW**	Accuracy: 0.79 Precession: 0.7924 Recall: 0.79 **F1 Score: 0.788**	Accuracy: 0.8525 Precession: 0.8511 Recall: 0.8525 **F1 Score: 0.8506**
	TF-IDF	Accuracy: 0.775 Precession: 0.8015 Recall: 0.775 F1 Score: 0.7696	Accuracy: 0.815 Precession: 0.8128 Recall: 0.815 F1 Score: 0.8075
Support Vector Machine	**BoW**	Accuracy: 0.8025 Precession: 0.8015 Recall: 0.8025 F1 Score: 0.8010	Accuracy: 0.8475 Precession: 0.8075 Recall: 0.8475 F1 Score: 0.8453
	TF-IDF	Accuracy: 0.83 Precession: 0.8323 Recall: 0.83 F1 Score: **0.8274**	Accuracy: 0.8675 Precession: 0.867 Recall: 0.8675 F1 Score: **0.8648**

This table shows the results of Naive Bayes, Logistic Regression, Support Vector Machine (SVM) and two text representation techniques: Bag of Words (BoW) and TF-IDF in both the datasets. To enable machine learning algorithms to work on text, it is converted into numerical features using BoW and TF-IDF. BoW: yield text based on word frequency, TF-IDF: yield informative words more important. Accuracy, Precision, Recall and F1 Score are used for evaluating the models.

When it comes to both the data sets, Naive Bayes: BoW outperforms TF-IDF. There is no difference between the accuracy of BoW on Dataset 1 and on Dataset 2.The accuracy of BoW is 0.825 on Dataset 1 and 0.8325 on Dataset 2. The accuracy of TF-IDF for Dataset 1 is 0.763 and for Dataset 2 is 0.6775. Therefore, in this experiment BoW is more effective for Naive Bayes.

Logistic Regression: BoW is performing better than TF-IDF as well. The accuracy for Dataset 1 with BoW and Dataset 2 (with BoW) is 0.7900 and 0.8525 respectively. The accuracy is 0.7750, and 0.8150, respectively, with TF-IDF. Hence, BoW is the best in Logistic Regression for Dataset 2.

Unlike the other two algorithms, Support Vector Machine (SVM) has better performance in TF-IDF. In the case of BoW, the accuracy of Dataset 1 and Dataset 2 are 0.8025 and 0.8475 respectively. The accuracies with TF-IDF are 0.8300 and 0.8675, respectively. When using Logistic Regression and SVM, we can see that Dataset 2 performs better in comparison to

Dataset 1. However, Naive Bayes (TF-IDF) is more effective for Dataset 1 as compared to Dataset 2.

Overall Best Result: SVM with TF-IDF on Dataset 2 gives the best result with the following parameters: Accuracy = 0.8675, Precision = 0.8670, Recall = 0.8675 and F1 Score = 0.8648.

Based on these results, one can conclude that the machine learning algorithms' performance depends on the chosen text representation technique. In this experiment, BoW performs better with Naive Bayes and Logistic regression while TF-IDF performs better with SVM. In general, SVM/TF-IDF on Dataset 2 has the most promising performance.

3 Selecting the Best BERT model

The main reason why the BERT model was chosen in this project is the fact that it has a state of the art bidirectional processing factor, and therefore can process a sequence of text in its entirety to capture subtle contextual cues. BERT already has a solid baseline of understanding of the language semantics, without fine-tuning, by transfer learning on its large pre-training on English corpora. This state-of-the-art architecture, namely its self-attention mechanism, is particularly beneficial in decoding dense, and specialised phrasing, including the terms used in the MSME Legal Dispute dataset, where conventional models based purely on the frequency of keywords tend to fail. Moreover, BERT will act as the ultimate standard in this experiment to compare both the traditional machine learning algorithms and the compressed variant, which is the DistilBERT. The entire BERT model enabled the project to empirically prove that deep architectural complexity is absolutely required to preserve high accuracy and stability as the project switches between news articles of normal size and highly complex, domain-specific text.

Model	Learning Rate	Batch Size	Weight Decay	Dataset 1				Dataset 2			
				Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
BERT	0.00002	16	0.01	0.8800	0.8830	0.8800	0.8785	0.7900	0.7906	0.7900	0.7891
	0.00003	16	0.01	0.8875	0.8917	0.8875	0.8867	0.700	0.7742	0.700	0.6516
	0.00002	32	0.01	0.8450	0.8566	0.8450	0.8461	0.8475	0.8556	0.8475	0.8468
	0.00003	32	0.01	0.8643	0.8512	0.8620	0.8565	0.8525	0.8698	0.8548	0.8536
	0.00002	16	0.1	0.8425	0.8440	0.8425	0.8424	0.8500	0.8584	0.8500	0.8496
	0.00003	16	0.1	0.8500	0.86155	0.8500	0.8492	0.8500	0.8615	0.8500	0.8492
	0.00002	32	0.1	0.8475	0.8544	0.8475	0.8480	0.8450	0.8531	0.8450	0.8444
	0.00003	**32**	**0.1**	**0.8650**	**0.8749**	**0.8650**	**0.8652**	**0.8650**	**0.8749**	**0.8650**	**0.8652**
DistilBERT	0.00002	16	0.01	0.800	0.7926	0.800	0.7924	0.4775	0.4134	0.4775	0.3702
	0.00003	16	0.01	0.830	0.8428	0.830	0.8314	0.5350	0.6171	0.5350	0.4851
	0.00002	32	0.01	0.750	0.7582	0.750	0.7244	0.3725	0.2343	0.3725	0.2373
	0.00003	32	0.01	0.810	0.8062	0.810	0.8033	0.4325	0.2626	0.4325	0.3247
	0.00002	16	0.1	0.835	0.8349	0.835	0.8329	0.4775	0.4130	0.4775	0.3700
	0.00003	**16**	**0.1**	**0.830**	**0.8428**	**0.830**	**0.8314**	**0.5300**	**0.6073**	**0.5300**	**0.4825**
	0.00002	32	0.1	0.750	0.7582	0.750	0.7244	0.3750	0.2384	0.3750	0.2386
	0.00003	32	0.1	0.810	0.8062	0.810	0.8033	0.4375	0.2661	0.4375	0.3290
ENSEMBLE				**0.8475**	**0.8509**	**0.8475**	**0.8469**	**0.5100**	**0.3799**	**0.5100**	**0.3970**

The best and most consistent predictive accuracy over diverse complexities of the two datasets. The best combination for BERT was learning rate of 0.00003, a higher batch size of 32, and a high weight decay of 0.1, which resulted in the most stable performance with an accuracy of about 86.5% and F1 score of ~86.5% on both datasets. This set up proved to be a great way to avoid performance loss.

4. Selecting the Best BERT variant model

The five most widely-used versions of BERT were identified for this project: RoBERTa, ELECTRA, ALBERT, DistilBERT and DeBERTa. Out of these, DistilBERT was chosen as the BERT model for the final experiments.

DistilBERT is chosen because of its balance of performance, model size and computational efficiency. Two text classification data sets: AG News and MSME Legal Dispute Classification are used in the project. The AG News is the general news articles and MSME is the set of documents pertaining to legal disputes. Both of these tasks rely on comprehending the meaning and context of a text, so a transformer-based model like DistilBERT should be used for these classification tasks.

To achieve the above, a smaller, faster version of BERT was created, which is known as DistilBERT, and is able to retain much of BERT's language understanding ability with fewer parameters. It is therefore not as costly to train and faster compared to the original BERT. An additional advantage of DistilBERT is that it can be compared with a full-sized BERT model since the project already contains the original BERT model.

The other variants were also investigated. Let's take in a bit more detail: RoBERTa is an optimized and improved version of BERT, and is generally higher cost on the compute side. A difference between the two is that ELECTRA is implemented differently – with a different pre-training approach, yet can lead to strong performance. ALBERT is more efficient in terms of the number of parameters, with parameter sharing, but its structure is more different from the standard BERT. DeBERTa is more computationally expensive, but supports some better attention mechanisms and can be very successful.

Given the project needs, the chosen datasets are small and training efficiency is a factor to consider along with the desire to compare the variant with the original BERT, DistilBERT provides a good balance between the two. Thus, DistilBERT was chosen as the BERT model to use in final term experiments.

Then the chosen DistilBERT model was fine-tuned with varying learning rates, batch sizes and weight decay values. Based on the two datasets, the configuration with the greatest Accuracy, Precision, Recall, and F1 Score was considered as the best configuration.

DistilBERT, a learning rate of 0.00003, a smaller batch size of 16, and a weight decay of 0.01 was chosen because it maximized the model's potential. While DistilBERT generally struggled with the second dataset, this specific configuration achieved the highest possible peak for the model, maintaining a solid 83% accuracy on Dataset 1 while mitigating the collapse on Dataset 2.

5 Ensemble Model

In the case of the last Ensemble model, optimal hyperparameters of each base model were chosen and aggregated through soft voting. Particularly, BERT was tweaked with a 0.00003 learning rate, a 32-batch size, and a weight decay of 0.1 whereas DistilBERT used a learning rate of 0.00003, a smaller batch size of 16, and a weight decay of 0.01.

Model	Dataset 1				Dataset 2			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Ensemble	0.8475	0.8509	0.8475	0.8469	0.5100	0.3799	0.5100	0.3970

However, the Ensemble model shows the comparison of both the datasets where the model shows a sharp contrast in the performances and thereby depicting an inconsistency in the reliability of the model's prediction. It does well on Dataset 1 with a high and well-balanced accuracy, precision, recall and F1 score of around 85%. But on Dataset 2, the performance of the model falls drastically with the overall accuracy of 51% and the very low F1-score of 39.70%. This strong degradation indicates that the second data text is very complex and the ensemble is sensitive to this feature.

6 Overall Result Analysis

Model	Dataset 1				Dataset 2			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
Naïve Bayes	0.83	0.83	0.83	0.84	0.83	0.83	0.83	0.83
Logistic Regression	0.79	0.7924	0.79	0.788	0.85	0.85	0.85	0.86
Support Vector Machine	0.83	0.832	0.83	0.83	0.86	0.86	0.86	0.87
BERT	**0.87**	**0.87**	**0.86**	**0.87**	**0.87**	**0.88**	**0.87**	**0.87**
DistilBERT	0.83	0.84	0.83	0.83	0.52	0.62	0.53	0.51
ENSEMBLE	0.85	0.85	0.85	0.85	0.51	0.34	0.51	0.40

From the experimental results, it is seen that the performance of the BERT model is far better than all the other models in all the standard AG News and complex MSME Legal Dispute datasets with a maximum accuracy of 0.88 and F1-scores of 0.88. We did, however, get some surprisingly good results with traditional machine learning models such as SVM (0.87 F1-score on legal text), which shows that an old-fashioned method of feature extraction, such as TF-IDF, is still very effective when dealing with a repetitive, domain-specific vocabulary. In contrast, DistilBERT was very poor with the legal data set (0.52 accuracy) and, due to poor predictions, in the Ensemble model combined with BERT, this lowered the overall accuracy drastically. I think that even if we had the resources to train our transformer models for 5 or more epochs, they would have performed much better and more consistently, as BERT, undoubtedly, provide the most powerful text classifying capabilities, but for the specific tasks, a simpler ML model is very useful and efficient in my opinion.

7 Limitations

- Restricted Computational Power: Limited access to compute power, such as the use of advanced GPUs, which can process and train large and resource-intensive transformer models like BERT efficiently.
- Limited Training Epochs: Deep learning models were trained for a minimal number of epochs because of time constraints and limited processing power, in the case of optimal models they are trained for 5 or more epochs.
- Effect in general performance: If there are no hardware restrictions, then the models present in this paper could be trained for a longer period, which would have enabled them to learn deeper patterns and converge to learn the appropriate contextual patterns, thus improving the accuracy and the end result of the ensemble model.

8 Conclusion

This project offered a thorough and intensive comparative study on text classification models, between the classical algorithms of Machine Learning and the most recent models of Transformers. Through the comparison of these architectures in two extremely different areas of interest, general news and very complex legal disputes, this research was able to determine the optimal conditions, hyperparameters, and architecture of the models needed to achieve the best predictive performance.

The results of the study indicate strongly that **BERT** is the best model in the natural language processing and presents its unmatched capacity to break down profound semantic connections among various data sets with an accuracy of up to 88%. But the real difference and worth of our work is its delicate discoveries. Instead of only establishing that deep learning is effective, our analysis found that the traditional models, especially **SVM**, can be incredibly powerful and computationally efficient with highly structured domain-specific vocabularies. Moreover, our systematic hyperparameter optimization has determined the precise parameter setups to drive these models to their logical extremes.

The critical transparency is the distinguishing characteristic of this project as a remarkable contribution to the field. Through the rigorous analysis of the performance failure of the DistilBERT on complex legal text and the diagnosis of the subsequent failure of the Ensemble model, our study is invaluably valuable in terms of providing a blueprint of deploying NLP in real-life applications. In a practical demonstration of ensembling, we empirically demonstrated that not all ensembling is a silver bullet and can in fact be very harmful when constituent models are not at the same level of optimisation.

This project is, after all, way beyond typical model benchmarking. It provides a very practical and well tested model that governs when to use lightweight traditional models and when to take advantage of computationally heavy transformers depending on the complexity of data and the resources that are available. The work is strong, insightful, and highly applicable to the field of data science and natural language processing, as it provides both highly accurate configurations and a clear insight on the architectural limitations.

Project Code

- **Data preprocessing code**

```
import pandas as pd
import numpy as np
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')



df1 = pd.read_excel('/content/AG_NEWS.xlsx')
df2 = pd.read_excel('/content/MSME_DATASET.xlsx')

df1.dropna(how='any', inplace=True)
df2.dropna(how='any', inplace=True)
text_col = 'text'
label_col = 'label'

X1 = df1[text_col].astype(str)
y1 = df1[label_col]

X2 = df2[text_col].astype(str)
y2 = df2[label_col]

display(df1.head(10))
display(df2.head(10))
def apply_tokenization(text):
   return word_tokenize(text)

tokens_1 = X1.apply(apply_tokenization)
tokens_2 = X2.apply(apply_tokenization)
```

```
print("Word found:Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Word found:Dataset 2 ", len(tokens_2))
print(tokens_2)
import pandas as pd
import numpy as np
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
def apply_case_folding(tokens):
    return [x.lower() for x in tokens]

tokens_1 = tokens_1.apply(apply_case_folding)
tokens_2 = tokens_2.apply(apply_case_folding)
print("Lower case:Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Lower case:Dataset 2 ", len(tokens_2))
print(tokens_2)
synonyms = {
    "won't": "will not",
    "can't": "cannot",
    "n't": "not",
    "good": "nice",
    "bad": "poor",
    "big": "large",
    "small": "little",
    "happy": "glad",
    "quick": "fast",
    "begin": "start",
    "prices" : "price"
}

def apply_synonym_substitution(tokens):
```

```
    replaced_tokens = []

    for t in tokens:
        if t in synonyms:
            t = synonyms[t]

        replaced_tokens.append(t)

    return replaced_tokens

tokens_1 = tokens_1.apply(apply_synonym_substitution)
tokens_2 = tokens_2.apply(apply_synonym_substitution)
print("Substitute Synonyms: Dataset 1 ", len(tokens_1))
print(tokens_1)
print("Substitute Synonyms: Dataset 2 ", len(tokens_2))
print(tokens_2)
stemmer = PorterStemmer()

def apply_stemming_and_join(tokens):
    stemmed_words = [stemmer.stem(word) for word in tokens]
    return " ".join(stemmed_words)

stokens_1 = tokens_1.apply(apply_stemming_and_join)
stokens_2= tokens_2.apply(apply_stemming_and_join)
print("Stems: Dataset 1 ", len(tokens_1))
print(stokens_1)
print("Stems: Dataset 2 ", len(tokens_2))
print(stokens_2)
import string

def apply_punctuation_removal(text):
    text_remove_mapping = str.maketrans("", "", string.punctuation)

    clean_text = text.translate(text_remove_mapping)

    return clean_text.split()

remp_1 = stokens_1.apply(apply_punctuation_removal)
remp_2 = stokens_2.apply(apply_punctuation_removal)

print("Punctuation Remove : Dataset 1", len(stokens_1))
print(remp_1.head())
print("\nPunctuation Remove : Dataset 2", len(stokens_2))
print(remp_2.head())
stop_words = set(stopwords.words("english"))
```

```
def apply_stopwords_removal(tokens):
    return [word for word in tokens if word not in stop_words]

X1_clean = remp_1.apply(apply_stopwords_removal)
X2_clean = remp_2.apply(apply_stopwords_removal)
print("Stop words remove: Dataset 1 ", len(X1_clean))
print(X1_clean)
print("Stop words remove: Dataset 2 ", len(X2_clean))
print(X2_clean)
x1_export_clean = remp_1.apply(lambda x: " ".join(x) if isinstance(x, list) else x)
x2_export_clean = remp_2.apply(lambda x: " ".join(x) if isinstance(x, list) else x)
y1 = df1[label_col]
y2 = df2[label_col]

clean_dataset_1_agnews = pd.DataFrame({
    "text": x1_export_clean,
    "label": y1
})

clean_dataset_2_legaldispute = pd.DataFrame({
    "text": x2_export_clean,
    "label": y2
})

# Export
clean_dataset_1_agnews.to_excel("clean_dataset_1_agnews.xlsx", index=False)
clean_dataset_2_legaldispute.to_excel("clean_dataset_2_legaldispute.xlsx",
index=False)
```

- **BERT code**

```
import pandas as pd
import numpy as np
import torch
from datasets import Dataset
from transformers import BertTokenizer, BertForSequenceClassification, Trainer,
TrainingArguments, EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from google.colab import drive

df1 = pd.read_excel("/content/clean_dataset_1_agnews.xlsx")
df1 = df1[['text', 'label']]

labels_list1 = df1['label'].unique().tolist()
```

```
label2id1 = {label: i for i, label in enumerate(labels_list1)}
df1['label'] = df1['label'].map(label2id1)

hf_dataset1 = Dataset.from_pandas(df1)
dataset_split1 = hf_dataset1.train_test_split(test_size=0.2, seed=42)
train_dataset1 = dataset_split1['train']
eval_dataset1 = dataset_split1['test']
num_labels1 = len(labels_list1)


df2 = pd.read_excel("/content/clean_dataset_1_agnews.xlsx")
df2 = df2[['text', 'label']]

labels_list2 = df2['label'].unique().tolist()
label2id2 = {label: i for i, label in enumerate(labels_list2)}
df2['label'] = df2['label'].map(label2id2)

hf_dataset2 = Dataset.from_pandas(df2)
dataset_split2 = hf_dataset2.train_test_split(test_size=0.2, seed=42)
train_dataset2 = dataset_split2['train']
eval_dataset2 = dataset_split2['test']
num_labels2 = len(labels_list2)

print(f"Dataset 1 - Train: {len(train_dataset1)}, Eval: {len(eval_dataset1)}")
print(f"Dataset 1 Labels Mapping: {label2id1}")

print(f"\nDataset 2 - Train: {len(train_dataset2)}, Eval: {len(eval_dataset2)}")
print(f"Dataset 2 Labels Mapping: {label2id2}")
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def tokenize_function(examples):
    return tokenizer(examples['text'], padding='max_length', truncation=True,
max_length=128)

tokenized_train1 = train_dataset1.map(tokenize_function, batched=True)
tokenized_eval1 = eval_dataset1.map(tokenize_function, batched=True)

tokenized_train2 = train_dataset2.map(tokenize_function, batched=True)
tokenized_eval2 = eval_dataset2.map(tokenize_function, batched=True)
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions,
average='weighted', zero_division=0)
    acc = accuracy_score(labels, predictions)
```

```
    return {'Accuracy': acc, 'Precision': precision, 'Recall': recall, 'F1': f1}
import math
from transformers import Trainer, TrainingArguments,
BertForSequenceClassification, EarlyStoppingCallback

bert_combinations = [
    {'lr': 0.00002, 'bs': 16, 'wd': 0.01},
    {'lr': 0.00003, 'bs': 16, 'wd': 0.01},
    {'lr': 0.00002, 'bs': 32, 'wd': 0.01},
    {'lr': 0.00003, 'bs': 32, 'wd': 0.01},
    {'lr': 0.00002, 'bs': 16, 'wd': 0.1},
    {'lr': 0.00003, 'bs': 16, 'wd': 0.1},
    {'lr': 0.00002, 'bs': 32, 'wd': 0.1},
    {'lr': 0.00003, 'bs': 32, 'wd': 0.1}
]

new_results_list = []

for combo in bert_combinations:
    lr, bs, wd = combo['lr'], combo['bs'], combo['wd']

    print(f"\n{'='*55}")
    print(f"BERT: LR={lr}, Batch={bs}, WD={wd} (Steps Only)")
    print(f"{'='*55}\n")

    print(">>> Training on Dataset 1...")
    model1 = BertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=num_labels1)

    args1 = TrainingArguments(
        output_dir=f'./BERT_Models/D1_lr{lr}_bs{bs}_wd{wd}',
        max_steps=100, learning_rate=lr,
        per_device_train_batch_size=bs, per_device_eval_batch_size=bs,
        weight_decay=wd, warmup_steps=10,
        eval_strategy="steps", eval_steps=25,
        save_strategy="steps", save_steps=25,
        load_best_model_at_end=True, metric_for_best_model="F1"
    )

    trainer1 = Trainer(model=model1, args=args1, train_dataset=tokenized_train1,
eval_dataset=tokenized_eval1, compute_metrics=compute_metrics,
callbacks=[EarlyStoppingCallback(early_stopping_patience=2)])
    trainer1.train()
    eval_results1 = trainer1.evaluate()
```

```
    print("\n>>> Training on Dataset 2...")
    model2 = BertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=num_labels2)

    args2 = TrainingArguments(
        output_dir=f'./BERT_Models/D2_lr{lr}_bs{bs}_wd{wd}',
        max_steps=100, learning_rate=lr,
        per_device_train_batch_size=bs, per_device_eval_batch_size=bs,
        weight_decay=wd, warmup_steps=10,
        eval_strategy="steps", eval_steps=25,
        save_strategy="steps", save_steps=25,
        load_best_model_at_end=True, metric_for_best_model="F1"
    )

    trainer2 = Trainer(model=model2, args=args2, train_dataset=tokenized_train2,
eval_dataset=tokenized_eval2, compute_metrics=compute_metrics,
callbacks=[EarlyStoppingCallback(early_stopping_patience=2)])
    trainer2.train()
    eval_results2 = trainer2.evaluate()

    new_results_list.append({
        'Model': 'BERT', 'Learning Rate': lr, 'Batch Size': bs, 'Weight Decay': wd,
        'Dataset 1 Acc': eval_results1['eval_Accuracy'], 'Dataset 1 Prec':
eval_results1['eval_Precision'],
        'Dataset 1 Rec': eval_results1['eval_Recall'], 'Dataset 1 F1':
eval_results1['eval_F1'],
        'Dataset 2 Acc': eval_results2['eval_Accuracy'], 'Dataset 2 Prec':
eval_results2['eval_Precision'],
        'Dataset 2 Rec': eval_results2['eval_Recall'], 'Dataset 2 F1':
eval_results2['eval_F1']
    })
```

- **BERT Variant code**

```
import pandas as pd
import numpy as np
import torch
from datasets import Dataset
from transformers import BertTokenizer, BertForSequenceClassification, Trainer,
TrainingArguments, EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from google.colab import drive
```

```
df1 = pd.read_excel("/content/clean_dataset_1_agnews.xlsx")
df1 = df1[['text', 'label']]

labels_list1 = df1['label'].unique().tolist()
label2id1 = {label: i for i, label in enumerate(labels_list1)}
df1['label'] = df1['label'].map(label2id1)

hf_dataset1 = Dataset.from_pandas(df1)
dataset_split1 = hf_dataset1.train_test_split(test_size=0.2, seed=42)
train_dataset1 = dataset_split1['train']
eval_dataset1 = dataset_split1['test']
num_labels1 = len(labels_list1)


df2 = pd.read_excel("/content/clean_dataset_2_legaldispute.xlsx")
df2 = df2[['text', 'label']]

labels_list2 = df2['label'].unique().tolist()
label2id2 = {label: i for i, label in enumerate(labels_list2)}
df2['label'] = df2['label'].map(label2id2)

hf_dataset2 = Dataset.from_pandas(df2)
dataset_split2 = hf_dataset2.train_test_split(test_size=0.2, seed=42)
train_dataset2 = dataset_split2['train']
eval_dataset2 = dataset_split2['test']
num_labels2 = len(labels_list2)

print(f"Dataset 1 - Train: {len(train_dataset1)}, Eval: {len(eval_dataset1)}")
print(f"Dataset 1 Labels Mapping: {label2id1}")

print(f"\nDataset 2 - Train: {len(train_dataset2)}, Eval: {len(eval_dataset2)}")
print(f"Dataset 2 Labels Mapping: {label2id2}")
from transformers import DistilBertTokenizer


tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

def tokenize_function(examples):

    return tokenizer(examples['text'], padding='max_length', truncation=True,
max_length=128)


tokenized_train1 = train_dataset1.map(tokenize_function, batched=True)
```

```
tokenized_eval1 = eval_dataset1.map(tokenize_function, batched=True)


tokenized_train2 = train_dataset2.map(tokenize_function, batched=True)

tokenized_eval2 = eval_dataset2.map(tokenize_function, batched=True)

print("DistilBERT Tokenization Complete!")

import math
from transformers import Trainer, TrainingArguments,
DistilBertForSequenceClassification

weight_decays = [0.01, 0.1]
batch_sizes = [16, 32]
learning_rates = [0.00002, 0.00003]

results_list = []

for wd in weight_decays:
    for bs in batch_sizes:
        for lr in learning_rates:
            print(f"\n==================================================
======")
            print(f"Training Combination: LR={lr}, Batch={bs}, WD={wd} (DistilBERT
- 1 Epoch)")
            print(f"==================================================
=====\n")

            print(">>> Training on Dataset 1...")

            model1 = DistilBertForSequenceClassification.from_pretrained('distilbert-
base-uncased', num_labels=num_labels1)

            total_steps1 = math.ceil(len(tokenized_train1) / bs) * 1
            warmup_steps1 = int(total_steps1 * 0.1)

            output_dir1 = f'./DistilBERT_Models/D1_lr{lr}_bs{bs}_wd{wd}'
            training_args1 = TrainingArguments(
                output_dir=output_dir1,
                num_train_epochs=1,
                learning_rate=lr,
                per_device_train_batch_size=bs,
                per_device_eval_batch_size=bs,
                weight_decay=wd,
```

```
        warmup_steps=warmup_steps1,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="F1"
    )

    trainer1 = Trainer(
        model=model1, args=training_args1,
        train_dataset=tokenized_train1, eval_dataset=tokenized_eval1,
        compute_metrics=compute_metrics
    )
    trainer1.train()
    eval_results1 = trainer1.evaluate()

    trainer1.save_model(f'{output_dir1}/best_model')

    print("\n>>> Training on Dataset 2...")

    model2 = DistilBertForSequenceClassification.from_pretrained('distilbert-
base-uncased', num_labels=num_labels2)

    total_steps2 = math.ceil(len(tokenized_train2) / bs) * 1
    warmup_steps2 = int(total_steps2 * 0.1)

    output_dir2 = f'./DistilBERT_Models/D2_lr{lr}_bs{bs}_wd{wd}'
    training_args2 = TrainingArguments(
        output_dir=output_dir2,
        num_train_epochs=1,
        learning_rate=lr,
        per_device_train_batch_size=bs,
        per_device_eval_batch_size=bs,
        weight_decay=wd,
        warmup_steps=warmup_steps2,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="F1"
    )

    trainer2 = Trainer(
        model=model2, args=training_args2,
        train_dataset=tokenized_train2, eval_dataset=tokenized_eval2,
        compute_metrics=compute_metrics
    )
    trainer2.train()
```

```
        eval_results2 = trainer2.evaluate()

        trainer2.save_model(f'{output_dir2}/best_model')

        results_list.append({
            'Model': 'DistilBERT',
            'Learning Rate': lr,
            'Batch Size': bs,
            'Weight Decay': wd,
            'Dataset 1 Acc': eval_results1['eval_Accuracy'],
            'Dataset 1 Prec': eval_results1['eval_Precision'],
            'Dataset 1 Rec': eval_results1['eval_Recall'],
            'Dataset 1 F1': eval_results1['eval_F1'],
            'Dataset 2 Acc': eval_results2['eval_Accuracy'],
            'Dataset 2 Prec': eval_results2['eval_Precision'],
            'Dataset 2 Rec': eval_results2['eval_Recall'],
            'Dataset 2 F1': eval_results2['eval_F1']
        })
```

- **Ensemble code.**

```
import pandas as pd
import numpy as np
import math
from datasets import Dataset
from scipy.special import softmax
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from transformers import (BertTokenizer, DistilBertTokenizer,
                          BertForSequenceClassification, DistilBertForSequenceClassification,
                          Trainer, TrainingArguments)

bert_lr = 0.00003
bert_bs = 32
bert_wd = 0.1

distil_lr = 0.00003
distil_bs = 16
distil_wd = 0.01

bert_tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
distil_tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

def run_ensemble_pipeline(file_path, dataset_name):
    print(f"\n{'='*60}")
    print(f"  STARTING PIPELINE FOR: {dataset_name}")
```

```
    print(f"{'='*60}\n")

    df = pd.read_excel(file_path)[['text', 'label']]
    labels_list = df['label'].unique().tolist()
    label2id = {label: i for i, label in enumerate(labels_list)}
    df['label'] = df['label'].map(label2id)
    num_labels = len(labels_list)

    dataset_split = Dataset.from_pandas(df).train_test_split(test_size=0.2, seed=42)
    train_dataset = dataset_split['train']
    eval_dataset = dataset_split['test']

    print(">>> Tokenizing data...")
    tokenized_train_bert = train_dataset.map(lambda e: bert_tokenizer(e['text'],
padding='max_length', truncation=True, max_length=128), batched=True)
    tokenized_eval_bert = eval_dataset.map(lambda e: bert_tokenizer(e['text'],
padding='max_length', truncation=True, max_length=128), batched=True)

    tokenized_train_distil = train_dataset.map(lambda e: distil_tokenizer(e['text'],
padding='max_length', truncation=True, max_length=128), batched=True)
    tokenized_eval_distil = eval_dataset.map(lambda e: distil_tokenizer(e['text'],
padding='max_length', truncation=True, max_length=128), batched=True)

    print(f"\n>>> Training BERT on {dataset_name}...")
    bert_model = BertForSequenceClassification.from_pretrained('bert-base-uncased',
num_labels=num_labels)

    training_args_bert = TrainingArguments(
        output_dir=f'./temp_bert_{dataset_name}',
        num_train_epochs=1,
        learning_rate=bert_lr,
        per_device_train_batch_size=bert_bs,
        per_device_eval_batch_size=bert_bs,
        weight_decay=bert_wd,
        eval_strategy="no",
        save_strategy="no",
        report_to="none"
    )

    trainer_bert = Trainer(
        model=bert_model,
        args=training_args_bert,
        train_dataset=tokenized_train_bert,
        eval_dataset=tokenized_eval_bert
    )
    trainer_bert.train()
```

```
    print(f">>> Generating BERT Predictions for {dataset_name}...")
    bert_preds = trainer_bert.predict(tokenized_eval_bert)
    bert_probs = softmax(bert_preds.predictions, axis=1)

    print(f"\n>>> Training DistilBERT on {dataset_name}...")
    distil_model = DistilBertForSequenceClassification.from_pretrained('distilbert-
base-uncased', num_labels=num_labels)

    training_args_distil = TrainingArguments(
        output_dir=f'./temp_distil_{dataset_name}',
        num_train_epochs=1,
        learning_rate=distil_lr,
        per_device_train_batch_size=distil_bs,
        per_device_eval_batch_size=distil_bs,
        weight_decay=distil_wd,
        eval_strategy="no",
        save_strategy="no",
        report_to="none"
    )

    trainer_distil = Trainer(
        model=distil_model,
        args=training_args_distil,
        train_dataset=tokenized_train_distil,
        eval_dataset=tokenized_eval_distil
    )
    trainer_distil.train()

    print(f">>> Generating DistilBERT Predictions for {dataset_name}...")
    distil_preds = trainer_distil.predict(tokenized_eval_distil)
    distil_probs = softmax(distil_preds.predictions, axis=1)

    print(f"\n>>> Ensembling (Soft Voting) for {dataset_name}...")
    ensemble_probs = (bert_probs + distil_probs) / 2
    ensemble_final_preds = np.argmax(ensemble_probs, axis=1)

    actual_labels = tokenized_eval_bert['label']
    precision, recall, f1, _ = precision_recall_fscore_support(actual_labels,
ensemble_final_preds, average='weighted', zero_division=0)
    acc = accuracy_score(actual_labels, ensemble_final_preds)

    print(f"\n{'='*50}")
    print(f"FINAL ENSEMBLE RESULTS: {dataset_name}")
    print(f"{'='*50}")
    print(f"Accuracy : {acc:.4f}")
```

```
    print(f"Precision: {precision:.4f}")
    print(f"Recall   : {recall:.4f}")
    print(f"F1 Score : {f1:.4f}")
    print(f"{'='*50}\n")
```

```
run_ensemble_pipeline(file_path="/content/clean_dataset_1_agnews.xlsx",
dataset_name="dataset_1_AGNews")

run_ensemble_pipeline(file_path="/content/clean_dataset_2_legaldispute.xlsx",
dataset_name="dataset_2_Legal")
```